Semana 9 · Entorno congelado, piloto y tamaño de muestra

Fase de la propuesta: campaña de mediciones (semanas 9-10).

Objetivo	Congelar el entorno de medición, ejecutar la prueba piloto en ambos modos del bus y calcular cuántas repeticiones necesita la campaña.
Dedicación estimada	5–7 horas (incluye dos reinicios y dos pilotos).
Antes de empezar	Semanas 1–8 cerradas.
Al terminar tendrán	`data/raw/piloto_*`, `data/processed/tamano_muestra.csv` y el número de repeticiones decidido.

Paso 1. Congelar y documentar el entorno

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
bash ../herramientas/verificar_entorno.sh
systemctl list-units --type=service --state=running > ../docs/evidencias/servicios_campana.txt
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
```

Decidan y anoten en `docs/decisiones.md`:

- **Gobernador de CPU**: recomendado `performance` durante la campaña (frecuencia fija = menos variación). Se aplica así y hay que repetirlo después de cada reinicio:

```
echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
```

- **Red**: si usan SSH por cable Ethernet pueden desactivar Wi-Fi y Bluetooth de la Raspberry (`dtoverlay=disable-wifi`, `dtoverlay=disable-bt` en config.txt). **Si entran por Wi-Fi, NO lo desactiven**: déjenlo igual en todas las mediciones.
- **Firmware congelado**: anoten el commit (`git rev-parse --short HEAD`) y el FRAME_SIZE. No cambien el firmware ni el código hasta terminar la campaña.
- **Durante las mediciones**: trabajen dentro de `tmux` y no escriban en la Raspberry (teclear por SSH también consume energía).

Paso 2. Piloto en modo estándar (100 kbit/s)

En config.txt: `dtoverlay=i2c3,pins_4_5,baudrate=100000`, reinicien y vuelvan a fijar el gobernador.

```
tmux new -s piloto
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 campana.py --bloque estandar --piloto --reps 10 --frame F
#   60 corridas = 2 grupos x 3 tamaños x 10. Salir de tmux sin detenerlo: Ctrl+b d
#   volver: tmux attach -t piloto
```

```
python3 resumen_rapido.py
```

Paso 3. Piloto en modo rápido (400 kbit/s)

Cambien a `baudrate=400000`, reinicien, fijen el gobernador y repitan con `--bloque rapido`.

Paso 4. Calcular el tamaño de muestra

```
python3 tamano_muestra.py ../data/raw/piloto_*_meta.json
```

El script aplica el criterio de la propuesta: `n = (100·z·s / (r·media))²` con r = 5 % y 95 % de confianza (Jain, 1991), con un mínimo de 30 (Montgomery & Runger, 2014). Muestra el `n_requerido` por métrica y la precisión que se lograría con 30, 60 y 100 repeticiones.

Paso 5. Decidir el número de repeticiones

- Estimen la duración de un bloque: `corridas = 6 × n`; `tiempo ≈ corridas × (duración media por corrida + 3 s de pausa) + 2 min de reposo` (la duración media la da `resumen_rapido.py`).
- Si `n_requerido` cabe en un tiempo razonable (por ejemplo, hasta ~3 h por bloque), úsenlo.
- Si alguna métrica (típicamente la energía por handshake) pide un n inalcanzable, fijen un n factible y **reporten la precisión lograda** con la tabla del script. Anoten la decisión con los números.

Revisión de calidad del piloto

Ejecuten `python3 analisis.py ../data/raw/piloto_*_meta.json` como ensayo general: deben generarse resumen, comparaciones y figuras. En `data/processed/corridas_con_energia.csv`, `muestras_en_hs` indica cuántas lecturas del INA219 caen dentro de cada handshake (más de 50 es cómodo). La deriva de reposo (`reposo_por_bloque.csv`) debería ser pequeña.

Criterio de cierre de la semana

- ☐ Entorno documentado (verificar_entorno, servicios, gobernador, red, commit del firmware).
- ☐ Pilotos estándar y rápido completos (60 corridas cada uno) sin fallas importantes.
- ☐ tamano_muestra.csv generado; número de repeticiones decidido y justificado en decisiones.md.
- ☐ analisis.py funcionó sobre el piloto (ensayo general).
- ☐ Commit y push.